Deterministic Timing Control on STM32F4:
Software Delays vs Hardware Timers and PWM

Objective

This report demonstrates the fundamental difference between software-based timing and hardware-based timing on an STM32F4 microcontroller. By comparing HAL_Delay() timing, timer interrupt scheduling, and hardware PWM generation, the report proves why hardware timers are mandatory for deterministic timing and real-time control systems.

System Clock and Timer Configuration

The STM32F446RE system clock was configured using the internal oscillator and PLL to achieve a SYSCLK of 84 MHz.

Clock Tree Summary

- SYSCLK = 84 MHz
- AHB prescaler = /1 → HCLK = 84 MHz
- APB1 prescaler = /2 → PCLK1 = 42 MHz

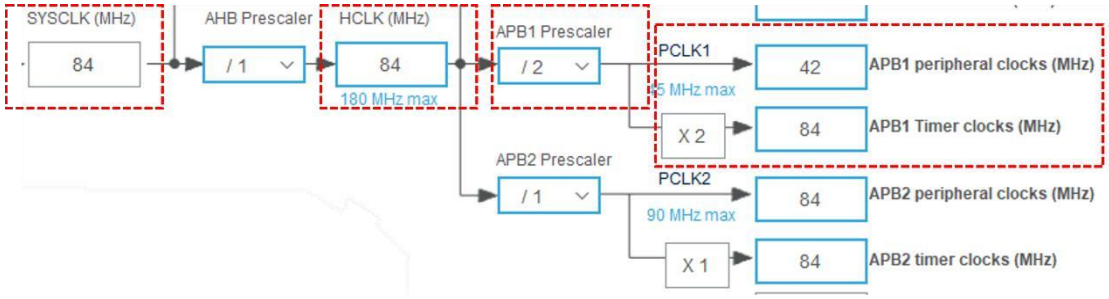


image 1

On STM32F4 devices, when the APB prescaler is greater than 1, the timer clock is automatically doubled. Therefore, TIM2 clock = 84 MHz as shown in image 1 above.

TIM2 Base Configuration

To generate a 1 ms time base:

- **Prescaler (PSC):** 83

$$\frac{84{,}000{,}000}{83+1} = 1{,}000{,}000 \text{ Hz}$$

- **Auto-Reload Register (ARR):** 999

$$\frac{999+1}{1{,}000{,}000} = 1 \text{ ms}$$

This produces a deterministic 1 ms timer period.

Timing Method Comparison

Method A: Software-Based Timing (Main Loop)

GPIO output was controlled using blocking delays inside the main execution loop.

```
/* USER CODE BEGIN Init */
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_SET);
HAL_Delay(1);
HAL_GPIO_WritePin(GPIOA, GPIO_PIN_5, GPIO_PIN_RESET);
HAL_Delay(1);
/* USER CODE END Init */
```

Result

Oscilloscope measurements showed a waveform of approximately **2 ms HIGH and 2 ms LOW**, significantly longer than the intended 1 ms timing as shown in image 2.

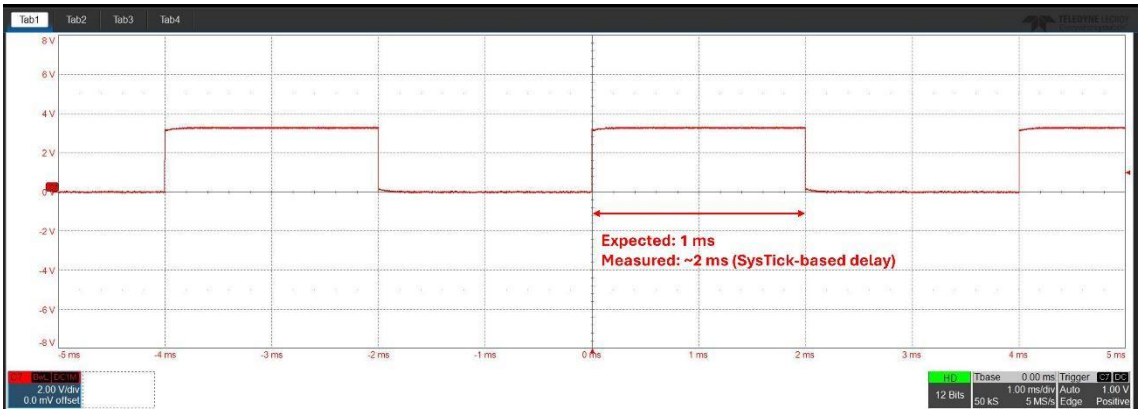


image 2

Root Cause:

HAL_Delay() is based on the SysTick timer, which updates at **1 ms resolution**. Each delay waits for the next SysTick boundary, resulting in a phase-dependent delay between **1 ms and nearly 2 ms per call**. Because two such delays are used per cycle, timing errors accumulate, producing non-deterministic behavior and jitter.

Method B: Hardware Timer Interrupt (TIM2)

GPIO output was toggled inside the TIM2 update interrupt callback.

```
void HAL_TIM_PeriodElapsedCallback(TIM_HandleTypeDef *htim)
{
    if (htim->Instance == TIM2)
    {
        HAL_GPIO_TogglePin(GPIOA, GPIO_PIN_5);
    }
}
```

Result

The oscilloscope showed a stable **500 Hz square wave**, corresponding to **1 ms HIGH and 1 ms LOW**. The waveform remained phase-locked on the screen, indicating deterministic timing as shown in image 3.

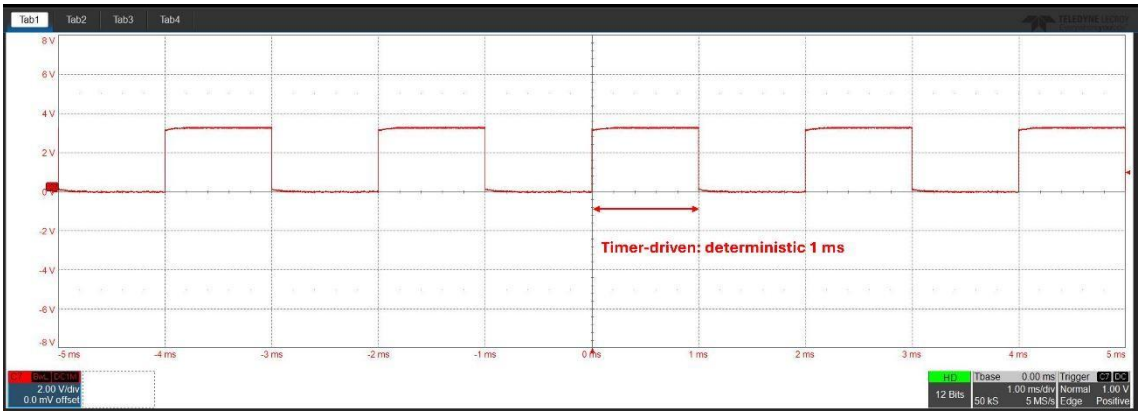


image 3

Explanation

The timer hardware generates update events independently of CPU execution. Interrupt latency introduces only a small, bounded delay that does not affect the timer period, resulting in stable periodic execution regardless of main-loop activity.

Hardware PWM Generation (No CPU Timing)

To eliminate interrupt latency entirely, TIM2 was configured in **PWM mode**.

PWM Configuration

- Timer: TIM2
- Channel: CH1
- GPIO Mapping: **PA5 configured as Alternate Function AF1 (TIM2_CH1)**
- Prescaler (PSC): 83
- Auto-Reload Register (ARR): 999
- Pulse (CCR): 500
- PWM Mode: PWM mode 1

In PWM mode 1:

- Output is HIGH while CNT < CCR
- Output is LOW while CNT ≥ CCR

This yields:

- Period = 1 ms (1 kHz)
- High time = 500 µs
- Low time = 500 µs
- Duty cycle = 50%

PWM Start Code

```
__HAL_TIM_SET_COMPARE(&htim2, TIM_CHANNEL_1, 500);
HAL_TIM_PWM_Start(&htim2, TIM_CHANNEL_1);
```

Result

The oscilloscope showed a **1 kHz PWM waveform** with a **50% duty cycle**, exhibiting no observable jitter or drift. Changing the CCR value modified the duty cycle without affecting frequency as shown below in image 4.

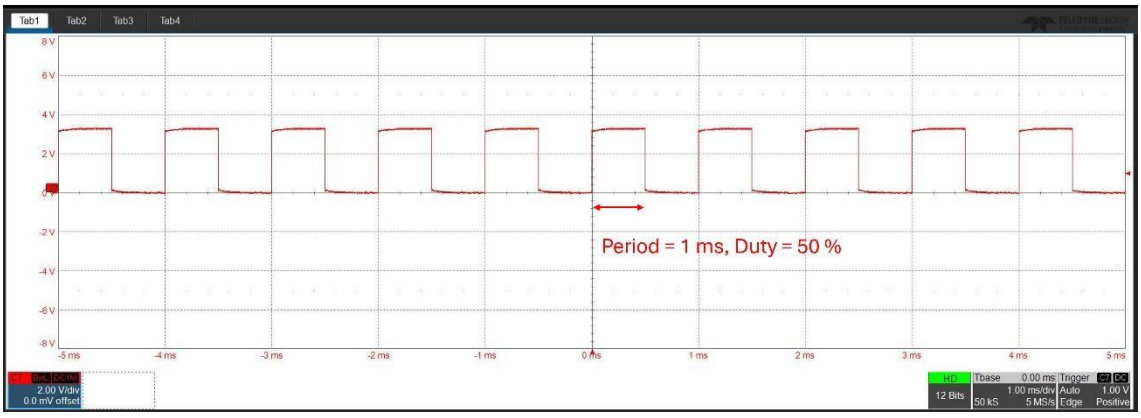


image 4

Key Point

Once configured, the PWM waveform is generated entirely by hardware. The CPU is not involved in edge timing.

Conclusion

Software-based delays such as HAL_Delay() are inherently non-deterministic due to SysTick quantization and blocking behavior. While acceptable for simple visual indicators, they are unsuitable for precise timing or control applications.

Timer interrupts provide deterministic periodic execution, but still incur interrupt latency. Hardware PWM eliminates CPU involvement entirely, offering the most stable and scalable solution for real-time systems.

For control systems, power electronics, and motor control applications, hardware timers and PWM peripherals are mandatory.